

Lab Activity: Artificial Neural Network (ANN)

Task

A simple Artificial Neural Network (ANN) to classify whether a student passes (1) or fails (0) based on two input features:

- study_hours
- sleep_hours

Dataset (20 records)

Study Hours	Sleep Hours	Pass (1)/Fail (0)
1	5	0
2	6	0
1	4	0
3	5	0
4	6	1
5	5	1
6	6	1
2	4	0
3	6	0
4	7	1
5	6	1
6	5	1
7	7	1
8	6	1
1	3	0
2	5	0
3	4	0
4	5	1
5	7	1
6	7	1

Artificial Neural Network (ANN) Example

Step 1: Import libraries

```
import numpy as np

from tensorflow.keras.models import Sequential

from tensorflow.keras.layers import Dense
```

Explanation:

- numpy (as np) → helps us work with numerical data in arrays and matrices.
- Sequential → used to create a linear stack of layers for our neural network (one layer after another).
- Dense → creates a fully connected neural network layer (each neuron connected to all neurons in the previous layer).

Step 2: Prepare the input data (features)

```
X = np.array([
    [1,5],[2,6],[1,4],[3,5],[4,6],
    [5,5],[6,6],[2,4],[3,6],[4,7],
    [5,6],[6,5],[7,7],[8,6],[1,3],
    [2,5],[3,4],[4,5],[5,7],[6,7]
])
```

Explanation:

- X is our input feature matrix — each row is one student.
- Each student has two features:
 1. Study hours
 2. Sleep hours
- For example, [1,5] means a student studied 1 hour and slept 5 hours.

So X has 20 rows (samples) and 2 columns (features).

Step 3: Prepare the output labels

```
y = np.array([
    0,0,0,0,1,
    1,1,0,0,1,
    1,1,1,1,0,
    0,0,1,1,1
])
```

Explanation:

- y is the target label (the result we want the ANN to predict).
- 0 = student failed, 1 = student passed.
- So, this is a binary classification problem (two classes: 0 or 1).

Step 4: Build the neural network

```
model = Sequential([
    Dense(8, input_dim=2, activation='relu'),
    Dense(4, activation='relu'),
    Dense(1, activation='sigmoid') # binary output
])
```

Explanation:

- `Sequential([...])` builds the ANN layer by layer.

Let's break it layer by layer:

1. `Dense(8, input_dim=2, activation='relu')`
 - First hidden layer
 - Has 8 neurons
 - `input_dim=2` → two input features (study hours, sleep hours)
 - `relu` (Rectified Linear Unit) → activation function that introduces non-linearity.
2. `Dense(4, activation='relu')`
 - Second hidden layer
 - Has 4 neurons
 - Again uses the ReLU activation function.
3. `Dense(1, activation='sigmoid')`
 - Output layer
 - Has 1 neuron because this is a binary classification (0 or 1).
 - `sigmoid` converts the output into a value between 0 and 1, representing the probability of passing.

Step 5: Compile the model

```
model.compile(optimizer='adam', loss='binary_crossentropy', metrics=['accuracy'])
```

Explanation:

- `optimizer='adam'` → the algorithm that adjusts weights to minimize the error (a popular choice).
- `loss='binary_crossentropy'` → used for binary classification problems (measures prediction error).
- `metrics=['accuracy']` → tells Keras to display accuracy during training and evaluation.

Step 6: Train the model

```
model.fit(X, y, epochs=100, verbose=0)
```

Explanation:

- `fit()` means train the model.

- `X = input data, y = correct labels.`
- `epochs=100` → the model goes through the entire dataset 100 times.
- `verbose=0` → hides training output (use 1 if you want to see the progress).

Step 7: Evaluate the trained model

```
loss, acc = model.evaluate(X, y, verbose=0)
```

```
print(f"Training Accuracy: {acc*100:.2f}%")
```

Explanation:

- `evaluate()` checks how well the model performs on the training data.
- It returns two values:
 - `loss` → how far predictions are from actual values.
 - `acc` → accuracy (how many were predicted correctly).
- The print line shows training accuracy in percentage format.
- `{acc*100:.2f}` takes the `acc` variable, multiplies by 100 (to get a percentage), and formats it to 2 decimal places (`.2f`).
- The `f` at the start tells Python to process these curly braces and replace them with the calculated value. Without `f`, it would just print the string literally:
-

Step 8: Test with new data

```
test = np.array([[3,5],[6,7],[2,3]])
```

```
pred = model.predict(test)
```

```
print("Predicted probabilities:", pred)
```

```
print("Predicted classes:", (pred > 0.5).astype(int))
```

Explanation:

- `test` → new unseen data (same format: `study_hours, sleep_hours`).
- `model.predict(test)` → gives output probabilities (values between 0 and 1).
- `(pred > 0.5).astype(int)` → converts probabilities into class labels:
 - `0.5` → Class 1 (Pass)
 - `≤ 0.5` → Class 0 (Fail)

Example Output

Training Accuracy: 95.00%

Predicted probabilities: [[0.32], [0.91], [0.08]]

Predicted classes: [[0], [1], [0]]

Meaning:

- Student with (3,5) $\rightarrow$ 0.32 $\rightarrow$ Fail
- Student with (6,7) $\rightarrow$ 0.91 $\rightarrow$ Pass
- Student with (2,3) $\rightarrow$ 0.08 $\rightarrow$ Fail
-

Interpretation:

Input (Study, Sleep)	Probability	Class	Meaning
(3,5)	0.32	0	Fail
(6,7)	0.91	1	Pass
(2,3)	0.08	0	Fail

9. Exercises for Students

1. **Change the number of epochs** (e.g., 50 $\rightarrow$ 200).
 - Observe how accuracy changes.
2. **Change the number of neurons** in hidden layers.
 - Example: `Dense(10, ...)` instead of `Dense(8, ...)`